

# Examen réparti 1 DataCloud Master 2 SAR

Jonathan Lejeune

Novembre 2024

2 heures – **Tout document papier interdit**  
**Tout appareil électronique interdit**

- Le barème est sur 21 points et est donné à titre indicatif. La note finale sera bornée à 20 points.
- Lors de la correction, il sera tenu compte de la simplicité et de la lisibilité de vos réponses
- S'abstenir de répondre est préférable à répondre une absurdité
- Vous trouverez en annexe de la documentation sur l'API de Spark (RDD + DStream)

## Exercice 1 – Questions de cours (6 points)

### Question 1 1 point

Lorsqu'un DataNode tombe en panne, expliquer

1. comment et par qui cette panne est détectée
2. quel est impact sur la disponibilité des données
3. le protocole exécuté par le système une fois la panne détectée

### Question 2 1 point

Même question que précédemment, mais pour le NameNode.

### Question 3 1 point

Pour chacun des types scala suivants, donner leur sémantique :

- Nothing
- Unit
- Null
- None

### Question 4 1 point

Donner la hiérarchie de sous-typage entre les 4 types de la question précédente.

### Question 5 1/2 point

Qu'affiche le programme suivant ? **Justifiez**<sup>1</sup>.

```
1 object TestPoint extends App{  
2   class Point(val x:Int, val y:Int)  
3   case class Point2(x:Int, y:Int)  
4 }
```

---

1. justification fausse ou absente  $\Rightarrow$  réponse fausse  $\Rightarrow$  pas de point

```

5  val p= new Point(0,0)
6  val q= new Point(0,0)
7  val p2 = Point2(0,0)
8  val q2 = Point2(0,0)
9  println(p==q)
10 println(p2==q2)
11 }

```

**Question 6**  $\frac{1}{2}$  point

Quelle(s) différence(s) y a-t-il entre un trait scala et une interface java ?

**Question 7**  $\frac{1}{2}$  point

Pourquoi Yarn est-il adapté pour exécuter des applications Spark ?

**Question 8**  $\frac{1}{2}$  point

Donnez le code de la fonction `curryfie[A,B,C](f: (A, B)=>C): A =>B =>C` qui renvoie une fonction qui est la version curryfiée de la fonction passée en paramètre

## Exercice 2 – Map-Reduce générique en Spark (6 points)

Nous proposons ci-dessous la méthode `reemapreduce` dans un `object Util`.

```

1  package datacloud.exam
2
3  object Util {
4    val hdfs = FileSystem.get(new Configuration)
5
6    def reemapreduce[K,V](sc:SparkContext,
7      pathin:Path,
8      mapfunction:(String,String)=>TraversableOnce[(K,V)],
9      reduceop:(V,V)=>V):RDD[(K,V)]={
10     if(hdfs.getFileStatus(pathin).isFile()){//est-ce un fichier régulier
11       val a = sc.textFile(pathin.toString())
12       val b = a.flatMap(mapfunction(_,pathin.toString()))
13       return b.reduceByKey(reduceop)
14     }else{//cas d'un répertoire
15       /*la méthode listStatus renvoie un Array[FileStatus].
16        * ce qui permet d'avoir la liste des fichiers dans le répertoire
17        */
18       val c = hdfs.listStatus(pathin).map(s=> s.getPath)
19       val d = c.map(p=>reemapreduce(sc, p, mapfunction, reduceop))
20       val e = d.reduce(_ union _)
21       return e.reduceByKey(reduceop,numPartitions=1)
22     }
23   }
24 }

```

Cette fonction permet de faire un traitement paramétrable récursivement sur une arborescence de fichiers textuels stockés sur HDFS (paramètre `pathin`) et de produire en sortie un résultat dans un `RDD[(K,V)]`, K et V étant des variables de type locales à la fonction. Le traitement de cette fonction est une composition de deux traitements renseignés en paramètre :

- `mapfunction` qui est une fonction qui pour une ligne de texte donnée et le nom de son fichier associé, permet de transformer cette ligne en une collection de couple clé-valeur de type respectif K et V.
- `reduceop` qui permet de définir un opérateur (associatif de réduction sur toutes les valeurs associées à une clé par la méthode `mapfunction`)

**Question 1** 1 point

Donner les types des variables `a`, `b`, `c`, `d` et `e`.

Soit le programme scala suivant

```

1 package datacloud.exam
2
3 import datacloud.exam.Util._
4
5 object MyApp extends App{
6   val conf = new SparkConf().setAppName("ExamWordcount")
7   val sc= new SparkContext(conf)
8
9   def wordcount(p:Path):RDD[(String,Int)] = {
10     //a completer
11   }
12
13   def index(p:Path):RDD[(String,Set[String])] = {
14     //a completer
15   }
16
17   wordcount(new Path("/in"))
18
19   index(new Path("/in2")).saveAsTextFile("/out2")
20 }
```

**Question 2** 1 point

En vous servant de la méthode `recoMapReduce`, donner le code de la fonction `wordcount` qui pour une arborescence de fichiers `Path` HDFS en passé en paramètre, renvoie un RDD associant à chaque mot son nombre d'occurrences dans l'ensemble des fichiers de l'arborescence.

**Question 3** 1 point

En vous servant de la méthode `recoMapReduce`, donner le code de la fonction `index` qui pour une arborescence de fichiers `Path` HDFS en passé en paramètre, renvoie un RDD associant à chaque mot l'ensemble des fichiers où ce mot a été trouvé. On prendra soin à ce qu'un même fichier ne soit pas listé plusieurs fois.

En tapant la commande `hdfs dfs -ls -h -R /` sur un terminal (l'option `-R` permet une lecture récursive d'une arborescence et l'option `-h` permet d'afficher la taille des fichiers de manière lisible), le terminal nous affiche la sortie suivante :

```

-rw-r--r--  1 jlejeune supergroup  801.2 M 2024-10-27 17:56 /in
drwxr-xr-x  - jlejeune supergroup      0 2024-10-27 17:57 /in2
-rw-r--r--  1 jlejeune supergroup  200.3 M 2024-10-27 17:57 /in2/f1
-rw-r--r--  1 jlejeune supergroup  200.3 M 2024-10-27 17:57 /in2/f2
drwxr-xr-x  - jlejeune supergroup      0 2024-10-27 17:57 /in2/f3
-rw-r--r--  1 jlejeune supergroup  200.3 M 2024-10-27 17:57 /in2/f3/f1
-rw-r--r--  1 jlejeune supergroup  200.3 M 2024-10-27 17:57 /in2/f3/f2
```

**Question 4**  $\frac{1}{2}$  point

Montrer que si on lance le programme MyApp tel quel, seul le traitement de `index` sera réellement exécuté.

**Question 5**  $2\frac{1}{2}$  points

Pour l'appel à `index` (ligne 19) :

1. dessiner le graphe de dépendance de RDD
2. y faire apparaître les stages
3. en déduire le graphe de tâche engendré par l'exécution de cette fonction.

## Exercice 3 – Matrices (6 points)

Le but de cet exercice est de programmer une classe scala `Matrix` permettant de manipuler des matrices d'entiers en se basant sur les RDD. Nous avons vu en TP, une implémentation basée sur des RDD de vecteur d'entier. Cependant cette implémentation stocke toutes les valeurs de la matrice ce qui n'est pas efficace si la matrice est creuse, i.e. si elle contient beaucoup de zéros. Notre nouvelle implantation va donc se baser sur un stockage "à plat" en associant chaque élément de la matrice à ses coordonnées et en ne stockant que les éléments non nuls. Nous allons définir l'état principal de la matrice par un attribut de type `RDD[Element]` où `Element` est défini ainsi :

```
1 case class Element(i:Long,j:Long,value:Int)
```

L'indice `i` (respectivement l'indice `j`) d'un élément est le numéro de sa ligne (resp. de sa colonne) et doit être compris entre 0 inclus et le nombre de lignes (res. de colonnes) exclu.

**Question 1** 4 points

Définir la classe `Matrix` qui contiendra :

- 3 attributs immuables :
  - ◇ `elements` de type `RDD[Element]`
  - ◇ `nbLin` de type `Long` pour le nombre total de lignes de la matrice
  - ◇ `nbCol` de type `Long` pour le nombre total de colonnes de la matrice
- 1 constructeur qui initialise ces trois attributs à partir de ses paramètres
- 3 méthodes :
  - ◇ `get` qui pour un numéro de ligne et un numéro de colonne donné, donne la valeur de l'élément. On supposera que les valeurs des deux paramètres sont valides.
  - ◇ `transpose` qui renvoie la transposée de la matrice.
  - ◇ `+` qui pour une `Matrix` donné renvoie une matrice qui résulte de l'addition de la matrice instance et de la matrice passée en paramètre. On supposera que les deux matrices ont le même nombre de lignes et le même nombre de colonnes. Pour rappel, l'addition de matrice revient à faire l'addition des éléments ayant les mêmes coordonnées.

**Question 2** 1 point

Dans un objet compagnon définir une fonction

```
makeMatrix(fich:String,sc:SparkContext,nbLin:Long,nbCol:Long)
```

qui construit une matrice à partir d'un fichier texte. On considérera que chaque ligne du fichier texte sera composée de trois entiers séparés par un espace : le premier pour le numéro de ligne, le second pour la colonne et le troisième pour la valeur. La fonction jettera une `IllegalArgumentException` si il existe un indice de colonne ou de ligne invalide.

**Question 3** 1 point

Dans ce même objet compagnon, définir une fonction

```
makeMatrix(mat:RDD[Array[Int]]):Matrix
```

qui construit une matrice à partir d'un RDD de vecteur d'entier. Vous prendrez soin de ne pas stocker les valeurs nulles.

## Exercice 4 – Monitoring de charge (3 points)

Un fournisseur de cloud dispose d'un ensemble de services accessibles à distance déployés sur un cluster de machines. A chaque service est dédié un ensemble de serveurs afin de paralléliser le traitement des requêtes. Toute requête de n'importe quel service aboutit sur une machine frontale du cluster qui fait office de load balancer. Le rôle de la machine frontale est d'aiguiller les requêtes vers les bons serveurs, i.e. les machines qui traiteront effectivement la requête du service demandé. Si l'ensemble des serveurs alloués à un service sont tous occupés à traiter des requêtes, le load balancer ajoute dans une file d'attente toute requête de ce service. Une requête sortira de sa file d'attente dès qu'un serveur se libère. Le load balancer maintient une file d'attente par service.

Notre but est de monitorer la charge de chaque service en temps réel afin de savoir s'il est nécessaire de redimensionner dynamiquement le nombre de serveurs alloués. Typiquement, un sous-dimensionnement pourra donner lieu au démarrage de nouveaux serveurs pour absorber la charge de requêtes, et un surdimensionnement pourra donner lieu à l'extinction de serveurs inutilisés dans un souci d'économie d'énergie. La quantification d'une charge d'un service donné se fera par la taille de sa file d'attente. Le load balancer est hébergé sur la machine *"lb.datacloud.com"* et émet un log toutes les secondes sur les connexions établies sur le port TCP 8794. Ce log contient trois champs séparés par un espace : le timestamp du log (en nombre de microsecondes depuis 1/1/1970), le nom du service et la taille de sa file d'attente.

**Question 1** 1½ points

Il faut environ une minute pour démarrer un nouveau serveur. Il est donc nécessaire d'avoir une vision de l'état des files d'attente sur au moins une minute. Écrire un programme spark qui affiche toutes les 10 secondes la taille moyenne de l'ensemble des files d'attente sur la dernière minute qui vient de s'écouler.

**Question 2** 1½ points

Nous souhaitons connaître l'accélération (ou la décélération) de la charge. Pour cela il est nécessaire de calculer le taux de variation de la charge. Pour rappel le taux de variation à l'instant  $t$  d'une valeur est la différence de valeur qu'il y a entre la valeur mesurée à l'instant  $t$  et la valeur mesurée à un instant précédent. Nous prendrons comme période de temps 10 secondes. Ainsi pour calculer le taux de variation, il faut faire la différence entre la valeur reçue à l'instant  $t$  soustraite à la valeur reçue il y a 10 secondes. Écrire un programme spark qui affiche toutes 10 secondes le taux de variation des 10 dernières secondes.

| API for SparkContext                                                                                                                                                              | Meaning                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| new <b>SparkContext</b> (config: SparkConf)                                                                                                                                       | Create a SparkContext that loads settings from a Spark Config object describing the application configuration. Any settings in this config overrides the default configs as well as system properties.                                                                                                                                                                                                                                              |
| new <b>SparkContext</b> (master: String, appName: String, conf: SparkConf)                                                                                                        | Alternative constructor that allows setting common Spark properties directly, : Cluster URL to connect to (e.g. mesos://host:port, spark://host:port, local[4]), a name for your application, to display on the cluster web UI and a SparkConf object specifying other Spark parameters                                                                                                                                                             |
| <b>textFile</b> (path: String, minPartitions: Int = defaultMinPartitions): RDD[String]                                                                                            | Read a text file from HDFS, a local file system (available on all nodes), or any Hadoop-supported file system URI, and return it as an RDD of Strings. Path is the path to the text file on a supported file system, minPartitions is the suggested minimum number of partitions for the resulting RDD. It returns RDD of lines of the text file                                                                                                    |
| <b>sequenceFile</b> [K, V](path: String, keyClass: Class[K], valueClass: Class[V]): RDD[(K, V)]                                                                                   | Get an RDD for a Hadoop SequenceFile with given key and value types. Path is the directory to the input data files, the path can be comma separated paths as a list of inputs, keyClass (resp. valueClass) is the Class of the key (resp. the value) associated with SequenceFileInputFormat. It returns RDD of tuples of key and corresponding value.                                                                                              |
| <b>parallelize</b> [T](seq: Seq[T], numSlices: Int ): RDD[T]                                                                                                                      | Distribute a local Scala collection to form an RDD. NumSlices is number of partitions to divide the collection into. It returns the RDD representing distributed collection                                                                                                                                                                                                                                                                         |
| <b>newAPIHadoopFile</b> [K, V, F <: InputFormat[K, V]]path: String, fClass: Class[F], kClass: Class[K], vClass: Class[V], conf: Configuration = hadoopConfiguration): RDD[(K, V)] | Get an RDD for a given Hadoop file with an arbitrary new API InputFormat and extra configuration options to pass to the input format. Path is the directory to the input data files, the path can be comma separated paths as a list of inputs, fclass is the storage format of the data to be read , kClass (resp. vClass) is the Class of the key (resp. the value) Hadoop configuration. It returns RDD of tuples of key and corresponding value |
| <b>broadcast</b> [T](value: T): Broadcast[T]                                                                                                                                      | Broadcast a read-only variable to the cluster, returning a org.apache.spark.broadcast.Broadcast object for reading it in distributed functions. The variable will be sent to each cluster only once.                                                                                                                                                                                                                                                |
| <b>collectionAccumulator</b> [T]: CollectionAccumulator[T]                                                                                                                        | Create and register a CollectionAccumulator, which starts with empty list and accumulates inputs by adding them into the list.                                                                                                                                                                                                                                                                                                                      |
| <b>doubleAccumulator</b> : DoubleAccumulator<br><b>longAccumulator</b> : LongAccumulator                                                                                          | Create and register a double or long accumulator, which starts with 0 and accumulates inputs by add.                                                                                                                                                                                                                                                                                                                                                |
| <b>register</b> (acc: AccumulatorV2[_, _]): Unit                                                                                                                                  | Register the given accumulator. Accumulators must be registered before use, or it will throw exception.                                                                                                                                                                                                                                                                                                                                             |
| <b>stop</b> (): Unit                                                                                                                                                              | Shut down the SparkContext.                                                                                                                                                                                                                                                                                                                                                                                                                         |

| Simple transformation for RDD[T]                                  | Meaning                                                                                                                                             |
|-------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>map</b> [U](f: (T) =>U): RDD[U]                                | Return a new distributed dataset formed by passing each element of the source through a function <i>f</i> .                                         |
| <b>mapValues</b> [U](f: V => U): RDD[(K, U)]                      | Pass each value in the key-value pair RDD through a map function without changing the keys this also retains the original RDD's partitioning.       |
| <b>filter</b> (f: (T) =>Boolean): RDD[T]                          | Return a new dataset formed by selecting those elements of the source on which <i>f</i> returns true.                                               |
| <b>flatMap</b> [U](f: (T) =>TraversableOnce[U]): RDD[U]           | Similar to map, but each input item can be mapped to 0 or more output items (so <i>f</i> should return a Seq rather than a single item).            |
| <b>flatMapValues</b> [U](f: V => TraversableOnce[U]): RDD[(K, U)] | Pass each value in the key-value pair RDD through a flatMap function without changing the keys ; this also retains the original RDD's partitioning. |

|                                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|--------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>union</b> (other: RDD[T]): RDD[T]       | Return a new dataset that contains the union of the elements in the source dataset and the argument.                                                                                                                                                                                                                                                                                                                                                                |
| <b>zip</b> [U](other: RDD[U]): RDD[(T, U)] | Zips this RDD with another one, returning key-value pairs with the first element in each RDD, second element in each RDD, etc. <i>Assumes that the two RDDs have the same number of partitions and the same number of elements in each corresponding partition</i> (e.g. one was made through a map on the other).                                                                                                                                                  |
| <b>zipWithIndex</b> (): RDD[(T, Long)]     | Zips this RDD with its element indices. The ordering is first based on the partition index and then the ordering of items within each partition. So the first item in the first partition gets index 0, and the last item in the last partition receives the largest index. This is similar to Scala's zipWithIndex but it uses Long instead of Int as the index type.<br>This method needs to trigger a spark job when this RDD contains more than one partitions. |
| <b>keyBy</b> [K](f: (T) =>K): RDD[(K, T)]  | Creates tuples of the elements in this RDD by applying f                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>glom</b> (): RDD[Array[T]]              | Return an RDD created by coalescing all elements within each partition into an array.                                                                                                                                                                                                                                                                                                                                                                               |

| Large transformation for RDD[T]                                                                                                                                                  | Meaning                                                                                                                                                      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>intersection</b> (other: RDD[T]): RDD[T]<br><b>intersection</b> (other: RDD[T], numPartitions: Int): RDD[T]<br><b>intersection</b> (other: RDD[T], part: Partitioner): RDD[T] | Return the intersection of this RDD and another one. The output will not contain any duplicate elements, even if the input RDDs did.                         |
| <b>distinct</b> (): RDD[T]<br><b>distinct</b> (numPartitions: Int): RDD[T]                                                                                                       | Return a new dataset that contains the distinct elements of the source dataset.                                                                              |
| <b>cartesian</b> [U](other: RDD[U]): RDD[(T, U)]                                                                                                                                 | When called on datasets of types T and U, returns a dataset of (T, U) pairs (all pairs of elements).                                                         |
| <b>repartition</b> (numPartitions: Int): RDD[T]                                                                                                                                  | Reshuffle the data in the RDD randomly to create either more or fewer partitions and balance it across them. This always shuffles all data over the network. |
| <b>subtract</b> (other: RDD[T]): RDD[T]<br><b>subtract</b> (other: RDD[T], numPartitions: Int): RDD[T]<br><b>subtract</b> (other: RDD[T], p: Partitioner): RDD[T]                | Return an RDD with the elements from <i>this</i> that are not in <i>other</i> .                                                                              |
| <b>sortBy</b> [K](f: (T) =>K, ascending: Boolean = true , numPartitions: Int = this.partitions.length): RDD[T]                                                                   | Return this RDD sorted by the given key function.                                                                                                            |

| Large transformation for RDD[T] and T= (K,V)                                                                                                                                                                | Meaning                                                                                                                                                                                                  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>groupByKey</b> (): RDD[(K, Iterable[V])]<br><b>groupByKey</b> (numPartitions:Int): RDD[(K, Iterable[V])]<br><b>groupByKey</b> (p:Partitioner): RDD[(K, Iterable[V])]                                     | When called on a dataset of (K, V) pairs, returns a dataset of (K, Iterable<V>) pairs.                                                                                                                   |
| <b>reduceByKey</b> (func: (V, V) =>V): RDD[(K, V)]<br><b>reduceByKey</b> (numPartitions:Int, func: (V, V) =>V): RDD[(K, V)]<br><b>reduceByKey</b> (partitioner: Partitioner, func: (V, V) =>V): RDD[(K, V)] | When called on a dataset of (K, V) pairs, returns a dataset of (K, V) pairs where the values for each key are aggregated using the given reduce function <i>func</i> , which must be of type (V,V) => V. |
| <b>sortByKey</b> (ascending: Boolean = true, numPartitions: Int = self.partitions.length) : RDD[(K, V)]                                                                                                     | When called on a dataset of (K, V) pairs where K implements Ordered, returns a dataset of (K, V) pairs sorted by keys in ascending or descending order, as specified in the boolean ascending argument.  |

|                                                                                                                                                                                                                                                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>join</b> [W](other: RDD[(K, W)]): RDD[(K, (V, W))]<br><b>join</b> [W](other: RDD[(K, W)], numPartitions: Int): RDD[(K, (V, W))]<br><b>join</b> [W](other: RDD[(K, W)], p: Partitioner): RDD[(K, (V, W))]                                                                                      | Return an RDD containing all pairs of elements with matching keys in <i>this</i> and <i>other</i> . Each pair of elements will be returned as a (k, (v1, v2)) tuple, where (k, v1) is in <i>this</i> and (k, v2) is in <i>other</i>                                                                                                                                                                                                                                                                             |
| <b>leftOuterJoin</b> [W](other: RDD[(K, W)]): RDD[(K, (V, Option[W]))]<br><b>leftOuterJoin</b> [W](other: RDD[(K, W)], numPartitions: Int): RDD[(K, (V, Option[W]))]<br><b>leftOuterJoin</b> [W](other: RDD[(K, W)], partitioner: Partitioner): RDD[(K, (V, Option[W]))]                         | Perform a left outer join of <b>this</b> and <b>other</b> . For each element (k, v) in <b>this</b> , the resulting RDD will either contain all pairs (k, (v, Some(w))) for w in <b>other</b> , or the pair (k, (v, None)) if no elements in <b>other</b> have key k.                                                                                                                                                                                                                                            |
| <b>rightOuterJoin</b> [W](other: RDD[(K, W)]): RDD[(K, (Option[V], W))]<br><b>rightOuterJoin</b> [W](other: RDD[(K, W)], numPartitions: Int): RDD[(K, (Option[V], W))]<br><b>rightOuterJoin</b> [W](other: RDD[(K, W)], partitioner: Partitioner): RDD[(K, (Option[V], W))]                      | Perform a right outer join of <b>this</b> and <b>other</b> . For each element (k, w) in <b>other</b> , the resulting RDD will either contain all pairs (k, (Some(v), w)) for v in <b>this</b> , or the pair (k, (None, w)) if no elements in <b>this</b> have key k.                                                                                                                                                                                                                                            |
| <b>fullOuterJoin</b> [W](other: RDD[(K, W)]): RDD[(K, (Option[V], Option[W]))]<br><b>fullOuterJoin</b> [W](other: RDD[(K, W)], numPartitions: Int): RDD[(K, (Option[V], Option[W]))]<br><b>fullOuterJoin</b> [W](other: RDD[(K, W)], partitioner: Partitioner): RDD[(K, (Option[V], Option[W]))] | Perform a full outer join of <b>this</b> and <b>other</b> . For each element (k, v) in <b>this</b> , the resulting RDD will either contain all pairs (k, (Some(v), Some(w))) for w in <b>other</b> , or the pair (k, (Some(v), None)) if no elements in <b>other</b> have key k. Similarly, for each element (k, w) in <b>other</b> , the resulting RDD will either contain all pairs (k, (Some(v), Some(w))) for v in <b>this</b> , or the pair (k, (None, Some(w))) if no elements in <b>this</b> have key k. |
| <b>cogroup</b> [W](o: RDD[(K, W)]): RDD[(K, (Iterable[V], Iterable[W]))]<br><b>cogroup</b> [W](o: RDD[(K, W)], numPart: Int): RDD[(K, (Iterable[V], Iterable[W]))]<br><b>cogroup</b> [W](o: RDD[(K, W)], p: Partitioner): RDD[(K, (Iterable[V], Iterable[W]))]                                   | When called on datasets of type (K, V) and (K, W), returns a dataset of (K, (Iterable<V>, Iterable<W>)) tuples. This operation is also called <b>groupWith</b> .                                                                                                                                                                                                                                                                                                                                                |
| <b>subtractByKey</b> [W](other: RDD[(K, W)]): RDD[(K, V)]<br><b>subtractByKey</b> [W](other: RDD[(K, W)], numPartitions: Int): RDD[(K, V)]<br><b>subtractByKey</b> [W](other: RDD[(K, W)], p: Partitioner): RDD[(K, V)]                                                                          | Return an RDD with the pairs from <i>this</i> whose keys are not in <i>other</i>                                                                                                                                                                                                                                                                                                                                                                                                                                |

| Action for RDD[T]                                          | Meaning                                                                                                                                                                                                              |
|------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>reduce</b> (f: (T, T) =>T): T                           | Aggregate the elements of the dataset using a function <i>func</i> (which takes two arguments and returns one). The function should be commutative and associative so that it can be computed correctly in parallel. |
| <b>collect</b> (): Array[T]                                | Return all the elements of the dataset as an array at the driver program. This is usually useful after a filter or other operation that returns a sufficiently small subset of the data.                             |
| <b>count</b> ():Long                                       | Return the number of elements in the dataset.                                                                                                                                                                        |
| <b>first</b> (): T                                         | Return the first element of the dataset (similar to take(1)).                                                                                                                                                        |
| <b>take</b> (num: Int): Array[T]                           | Return an array with the first <i>n</i> elements of the dataset.                                                                                                                                                     |
| <b>top</b> (num: Int)(implicit ord: Ordering[T]): Array[T] | Returns the top k (largest) elements from this RDD as defined by the specified implicit Ordering[T] and maintains the ordering.                                                                                      |
| <b>saveAsObjectFile</b> (path: String): Unit               | Save this RDD as a SequenceFile of serialized objects.                                                                                                                                                               |
| <b>saveAsTextFile</b> (path: String): Unit                 | Save this RDD as a text file, using string representations of elements.                                                                                                                                              |
| <b>foreach</b> (f: (T) =>Unit): Unit                       | Run a function <i>func</i> on each element of the dataset. This is usually done for side effects such as updating an <a href="#">Accumulator</a> or interacting with external storage systems.                       |

| API for StreamingContext                                                                                                                             | Meaning                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| new <b>StreamingContext</b> (sparkContext: SparkContext, batchDuration: Duration)                                                                    | Create a StreamingContext using an existing SparkContext. <i>BatchDuration</i> is the time interval at which streaming data will be divided into batches. Duration can be initialized by <i>Milliseconds(nb)</i> , <i>Seconds(nb)</i> or <i>Minutes(nb)</i>                                                                                                                                                                                                                                                                                       |
| <b>checkpoint</b> (directory: String)                                                                                                                | Set the context to periodically checkpoint the DStream operations for driver fault-tolerance. The directory is HDFS-compatible directory where the checkpoint data will be reliably stored. Note that this must be a fault-tolerant file system like HDFS.                                                                                                                                                                                                                                                                                        |
| <b>socketTextStream</b> ( hostname: String, port: Int, sl: StorageLevel = StorageLevel.MEMORY_AND_DISK_SER_2 ): ReceiverInputDStream[String]         | Creates an input stream from TCP source hostname:port. Data is received using a TCP socket and the receive bytes is interpreted as UTF8 encoded <code>`n`</code> delimited lines. The <i>hostname</i> parameter is the host to connect to for receiving data, <i>port</i> is the port to connect to for receiving data, <i>sl</i> is the Storage level to use for storing the received objects.                                                                                                                                                   |
| <b>socketStream</b> [T: ClassTag]( hostname: String, port: Int, converter: (InputStream) => Iterator[T], sl: StorageLevel ): ReceiverInputDStream[T] | Create an input stream from network source hostname:port, where data is received as serialized blocks (serialized using the Spark's serializer) that can be directly pushed into the block manager without deserializing them. This is the most efficient way to receive data. The <i>hostname</i> parameter is the host to connect to for receiving data, <i>port</i> is the port to connect to for receiving data, <i>sl</i> is the Storage level to use for storing the received objects. T is the type of the objects in the received blocks. |
| <b>textFileStream</b> (directory: String): DStream[String]                                                                                           | Create an input stream that monitors a Hadoop-compatible filesystem for new files and reads them as text files (using key as LongWritable, value as Text and input format as TextInputFormat). Files must be written to the monitored directory by "moving" them from another location within the same file system. File names starting with <code>.</code> are ignored.                                                                                                                                                                          |
| <b>start</b> () : Unit                                                                                                                               | Start the execution of the streams. Throws <code>IllegalStateException</code> if the StreamingContext is already stopped.                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>awaitTermination</b> ()                                                                                                                           | Wait for the execution to stop. Any exceptions that occurs during the execution will be thrown in this thread.                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>awaitTerminationOrTimeout</b> (timeout: Long): Boolean                                                                                            | Wait for the execution to stop. Any exceptions that occurs during the execution will be thrown in this thread. The <i>timeout</i> parameter is the time to wait in milliseconds. Returns true if it's stopped; or throw the reported error during the execution; or false if the waiting time elapsed before returning from the method.                                                                                                                                                                                                           |
| <b>stop</b> ( stopSparkContext: Boolean = conf.getBoolean("spark.streaming.stopSparkContextByDefault", true) ): Unit                                 | Stop the execution of the streams immediately (does not wait for all received data to be processed). By default, if <i>stopSparkContext</i> is not specified, the underlying SparkContext will also be stopped. This implicit behavior can be configured using the SparkConf configuration <i>spark.streaming.stopSparkContextByDefault</i> . If <i>stopSparkContext</i> is true, it stops the associated SparkContext. The underlying SparkContext will be stopped regardless of whether this StreamingContext has been started.                 |

| Transformation for DStream[T]                                                                                                                                                          | Meaning                                                                                                                                                                                                                                                                                                     |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>filter</b> (filterFunc: (T) => Boolean): DStream[T]                                                                                                                                 | Return a new DStream containing only the elements that satisfy a predicate.                                                                                                                                                                                                                                 |
| <b>map</b> [U](mapFunc: (T) => U): DStream[U]                                                                                                                                          | Return a new DStream by applying a function to all elements of this DStream                                                                                                                                                                                                                                 |
| <b>flatMap</b> [U](flatMapFunc: (T) => TraversableOnce[U]): DStream[U]                                                                                                                 | Return a new DStream by applying a function to all elements of this DStream, and then flattening the results                                                                                                                                                                                                |
| <b>glom</b> () : DStream[Array[T]]                                                                                                                                                     | Return a new DStream in which each RDD is generated by applying <code>glom()</code> to each RDD of this DStream. Applying <code>glom()</code> to an RDD coalesces all elements within each partition into an array.                                                                                         |
| <b>union</b> (that: Dstream[T]): DStream[T]                                                                                                                                            | Return a new DStream by unifying data of another DStream with this DStream.                                                                                                                                                                                                                                 |
| <b>repartition</b> (numPartitions: Int): DStream[T]                                                                                                                                    | Return a new DStream with an increased or decreased level of parallelism. Each RDD in the returned DStream has exactly <code>numPartitions</code> partitions.                                                                                                                                               |
| <b>count</b> () : DStream[Long]                                                                                                                                                        | Return a new DStream in which each RDD has a single element generated by counting each RDD of this DStream.                                                                                                                                                                                                 |
| <b>countByValue</b> (numPartitions: Int = ssc.sc.defaultParallelism)(implicit ord: Ordering[T] = null): DStream[(T, Long)]                                                             | Return a new DStream in which each RDD contains the counts of each distinct value in each RDD of this DStream. Hash partitioning is used to generate the RDDs with <code>numPartitions</code> partitions (Spark's default number of partitions if <code>numPartitions</code> not specified).                |
| <b>countByValueAndWindow</b> (windowDuration: Duration, slideDuration: Duration, numPartitions: Int = ssc.sc.defaultParallelism)(implicit ord: Ordering[T] = null): DStream[(T, Long)] | Return a new DStream in which each RDD contains the count of distinct elements in RDDs in a sliding window over this DStream. Hash partitioning is used to generate the RDDs with <code>numPartitions</code> partitions (Spark's default number of partitions if <code>numPartitions</code> not specified). |
| <b>transform</b> [U](transformFunc: (RDD[T], Time) => RDD[U])(implicit arg0: ClassTag[U]): DStream[U]                                                                                  | Return a new DStream in which each RDD is generated by applying a function on each RDD of 'this' DStream.                                                                                                                                                                                                   |
| <b>reduce</b> (reduceFunc: (T, T) => T): DStream[T]                                                                                                                                    | Return a new DStream in which each RDD has a single element generated by reducing each RDD of this DStream.                                                                                                                                                                                                 |
| <b>foreachRDD</b> (foreachFunc: (RDD[T]) => Unit): Unit                                                                                                                                | Apply a function to each RDD in this DStream. This is an output operator, so 'this' DStream will be registered as an output stream and therefore materialized.                                                                                                                                              |
| <b>countByWindow</b> (windowDuration: Duration, slideDuration: Duration): DStream[Long]                                                                                                | Return a new DStream in which each RDD has a single element generated by counting the number of elements in a sliding window over this DStream. Hash partitioning is used to generate the RDDs with Spark's default number of partitions                                                                    |
| <b>reduceByWindow</b> (reduceFunc: (T, T) => T, windowDuration: Duration, slideDuration: Duration): DStream[T]                                                                         | Return a new DStream in which each RDD has a single element generated by reducing all elements in a sliding window over this DStream.                                                                                                                                                                       |
| <b>window</b> (windowDuration: Duration, slideDuration: Duration): DStream[T]                                                                                                          | Return a new DStream in which each RDD contains all the elements in seen in a sliding window of time over this DStream.                                                                                                                                                                                     |

| Transformation for DStream[T], T= (K,V)                                                                                                                                           | Meaning                                                                                                                                                                                                                                                                                                                                            |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>combineByKey</b> [C](createCombiner: (V) => C, mergeValue: (C, V) => C, mergeCombiner: (C, C) => C, partitioner: Partitioner, mapSideCombine: Boolean = true): DStream[(K, C)] | Combine elements of each key in DStream's RDDs using custom functions. This is similar to the combineByKey for RDDs. Please refer to combineByKey in org.apache.spark.rdd.PairRDDFunctions in the Spark core documentation for more information.                                                                                                   |
| <b>reduceByKey</b> (reduceFunc: (V, V) => V): DStream[(K, V)]                                                                                                                     | Return a new DStream by applying <b>reduceByKey</b> to each RDD. The values for each key are merged using the associative and commutative reduce function. Hash partitioning is used to generate the RDDs with Spark's default number of partitions.                                                                                               |
| <b>mapValues</b> [U](mapValuesFunc: (V) => U): DStream[(K, U)]                                                                                                                    | Return a new DStream by applying a map function to the value of each key-value pairs in 'this' DStream without changing the key                                                                                                                                                                                                                    |
| <b>flatMapValues</b> [U](flatMapValuesFunc: (V) => TraversableOnce[U]): DStream[(K, U)]                                                                                           | Return a new DStream by applying a flatmap function to the value of each key-value pairs in 'this' DStream without changing the key.                                                                                                                                                                                                               |
| <b>join</b> [W](other: DStream[(K, W)]): DStream[(K, (V, W))]                                                                                                                     | Return a new DStream by applying 'join' between RDDs of <b>this</b> DStream and <b>other</b> DStream. Hash partitioning is used to generate the RDDs with Spark's default number of partitions.                                                                                                                                                    |
| <b>leftOuterJoin</b> [W](other: DStream[(K, W)]): DStream[(K, (V, Option[W]))]                                                                                                    | Return a new DStream by applying 'left outer join' between RDDs of <b>this</b> DStream and <b>other</b> DStream.                                                                                                                                                                                                                                   |
| <b>fullOuterJoin</b> [W](other: DStream[(K, W)]): DStream[(K, (Option[V], Option[W]))]                                                                                            | Return a new DStream by applying 'full outer join' between RDDs of <b>this</b> DStream and <b>other</b> DStream. Hash partitioning is used to generate the RDDs with Spark's default number of partitions.                                                                                                                                         |
| <b>rightOuterJoin</b> [W](other: DStream[(K, W)]): DStream[(K, (Option[V], W))]                                                                                                   | Return a new DStream by applying 'right outer join' between RDDs of <b>this</b> DStream and <b>other</b> DStream. Hash partitioning is used to generate the RDDs with Spark's default number of partitions.                                                                                                                                        |
| <b>cogroup</b> [W](other: DStream[(K, W)])(implicit arg0: ClassTag[W]): DStream[(K, (Iterable[V], Iterable[W]))]                                                                  | Return a new DStream by applying 'cogroup' between RDDs of <b>this</b> DStream and <b>other</b> DStream. Hash partitioning is used to generate the RDDs with Spark's default number of partitions.                                                                                                                                                 |
| <b>groupByKey</b> (): DStream[(K, Iterable[V])]                                                                                                                                   | Return a new DStream by applying <b>groupByKey</b> to each RDD. Hash partitioning is used to generate the RDDs with Spark's default number of partitions.                                                                                                                                                                                          |
| <b>groupByKeyAndWindow</b> (windowDuration: Duration): DStream[(K, Iterable[V])]                                                                                                  | Return a new DStream by applying <b>groupByKey</b> over a sliding window. This is similar to <b>DStream.groupByKey()</b> but applies it over a sliding window. The new DStream generates RDDs with the same interval as this DStream. Hash partitioning is used to generate the RDDs with Spark's default number of partitions.                    |
| <b>reduceByKeyAndWindow</b> (reduceFunc: (V, V) => V, windowDuration: Duration): DStream[(K, V)]                                                                                  | Return a new DStream by applying <b>reduceByKey</b> over a sliding window on <b>this</b> DStream. Similar to <b>DStream.reduceByKey()</b> , but applies it over a sliding window. The new DStream generates RDDs with the same interval as this DStream. Hash partitioning is used to generate the RDDs with Spark's default number of partitions. |

|                                                                                                                                                                                                                                                                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>reduceByKeyAndWindow</b> (<br>reduceFunc: (V, V) => V,<br>invReduceFunc: (V, V) => V,<br>windowDuration: Duration,<br>slideDuration: Duration = self.slideDuration,<br>numPartitions: Int = ssc.sc.defaultParallelism,<br>filterFunc: ((K, V)) => Boolean = null<br>): DStream[(K, V)] | Return a new DStream by applying incremental <code>`reduceByKey`</code> over a sliding window. The reduced value of over a new window is calculated using the old window's reduced value : <ol style="list-style-type: none"> <li>reduce the new values that entered the window (e.g., adding new counts)</li> <li>"inverse reduce" the old values that left the window (e.g., subtracting old counts)</li> </ol> This is more efficient than <code>reduceByKeyAndWindow</code> without "inverse reduce" function. However, it is applicable to only "invertible reduce functions". Hash partitioning is used to generate the RDDs with Spark's default number of partitions. |
| <b>updateStateByKey</b> [S](updateFunc: (Seq[V], Option[S]) => Option[S]):DStream[(K,S)]                                                                                                                                                                                                  | Return a new "state" DStream where the state for each key is updated by applying the given function on the previous state of the key and the new values of each key. In every batch the <code>updateFunc</code> will be called for each state even if there are no new values. Hash partitioning is used to generate the RDDs with Spark's default number of partitions. If <code>updateFunc</code> function returns <code>None</code> , then corresponding state key-value pair will be eliminated.                                                                                                                                                                          |

| Output Operation for DStream[T]                                                                | Meaning                                                                                                                                                                                                      |
|------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>foreachRDD</b> (foreachFunc: (RDD[T]) => Unit): Unit                                        | Apply a function to each RDD in this DStream. This is an output operator, so 'this' DStream will be registered as an output stream and therefore materialized.                                               |
| <b>print</b> (num: Int): Unit                                                                  | Print the first num elements of each RDD generated in this DStream. This is an output operator, so this DStream will be registered as an output stream and there materialized.                               |
| <b>print</b> (): Unit                                                                          | Print the first ten elements of each RDD generated in this DStream. This is an output operator, so this DStream will be registered as an output stream and there materialized.                               |
| <b>saveAsObjectFiles</b> (prefix: String, suffix: String = ""): Unit                           | Save each RDD in this DStream as a Sequence file of serialized objects. The file name at each batch interval is generated based on <code>prefix</code> and <code>suffix</code> : "prefix-TIME_IN_MS.suffix". |
| <b>saveAsTextFiles</b> (prefix: String, suffix: String = ""): Unit                             | Save each RDD in this DStream as at text file, using string representation of elements.                                                                                                                      |
| <b>saveAsNewAPIHadoopFiles</b> [F <: OutputFormat[K, V]](prefix: String, suffix: String): Unit | Save each RDD in <b>this</b> DStream as a Hadoop file. The file name at each batch interval is generated based on <code>prefix</code> and <code>suffix</code> : "prefix-TIME_IN_MS.suffix".                  |